

Loading Block Activity Guide — four ways to land files in BigQuery

Your guide for the Day 1 (Mon Oct 5, 2026) afternoon loading block, 13:45–14:30 CT. The instructor demonstrates each activity, then you do it yourself. All four activities come from the [Do-It-Now \(DIN\) activities page](#) (§24–§27), and all four are free — batch loads cost nothing. The concepts behind them are in the [Module 3 notes](#).

What you'll do

Four short activities, in order:

1. **§24 — Console & CLI load.** Load Monday's 1M-row vendor file with the console, append Tuesday's with the bq CLI.
2. **§25 — The autodetect pitfall.** Watch schema autodetect fail two different ways on dirty vendor data — once loudly, once silently — then fix it with a pinned schema.
3. **§26 — CSV vs Parquet race.** Load the same 60 million rows twice: once as one gzipped CSV (~2 minutes), once as Parquet (seconds).
4. **§27 — LOAD DATA in SQL.** Do the activity-1 loads again as pure SQL statements with an explicit schema.

Why it matters for your migration

This is your Bronze-landing pattern, hands-on: vendor file → Cloud Storage bucket → BigQuery table — the same shape as the legacy extract jobs you're replacing. Along the way you'll pick up the four rules that keep a Bronze layer honest: pinned schemas, the right file formats, free batch loads instead of row-by-row inserts, and everything expressible as repeatable SQL that lives in source control and a scheduler.

Before you start

- ☐ You created a dataset named **class** in this morning's exploring-datasets activity (DIN §3). Everything in this block loads into class. If you don't have it, create it now: BigQuery console → your project → Create dataset.
- ☐ The **BigQuery console** is open, and the project picker at the top shows **your lab project**.
- ☐ You know how to open **Cloud Shell** (the terminal icon at the top right of the console).

The two asset files you need are in this folder: `autodetect_demo.sh` (activity 2) and `load_data_example.sql` (activity 4). They are byte-identical copies of the DIN page's source files ([shell source](#), [SQL source](#)), so you can work from the local copies even if the network is slow.

Common gotchas — the whole block

- **Wrong project picker.** If the console shows a different project than your lab project, your tables land where you can't find them. Check the picker first, every time.

- **Cloud Shell first-run clicks.** The first time you open Cloud Shell you may get consent/authorize screens — click through them (Continue/Authorize) and wait for the shell prompt.
- **API-enable prompts.** A fresh lab project sometimes asks to enable an API the first time you touch a service. Click **Enable** and carry on.
- **Cloud Shell is scratch space.** Files you create there (activity 2) live on a temporary VM, not in your bucket. That's all these activities need — just don't expect them to survive the session.

Activity 1 (DIN §24) — Loading data: console and CLI

What you'll do. A vendor drops a usage file into a Cloud Storage bucket every day. You'll load Monday's file (usage_2026-07-20.csv, 1,000,000 rows) into a new table with the console, then append Tuesday's file with the CLI.

Why it matters. The console form and the bq command are two front ends over the same job API — the thing you'll script later is the thing you click today. This is the extract-job replacement in miniature.

Steps.

1. In the **Explorer** panel, click the three-dot menu next to your **class** dataset and select **Create table**.
2. Set **Create table from** to **Google Cloud Storage**.
3. In the file field, enter jwd-gcp-demos/ingest_demo/daily/usage_2026-07-20.csv — **without** the gs:// prefix (the console adds it).
4. Confirm **File format** is **CSV**. Name the table usage.
5. In the **Schema** section, check **Auto detect**, click **Create table**, and wait for the load to finish.
6. Open the new usage table's **Details** tab: **Number of rows** is **1,000,000**. Check the **Schema** tab: minutes came out **INTEGER** and event_date came out **DATE** — autodetect chose well *this time*. Remember that phrase.
7. Tuesday's file has arrived. Open **Cloud Shell** and append it to the same table:

```
bq load --autodetect --source_format=CSV \
class.usage \
gs://jwd-gcp-demos/ingest_demo/daily/usage_2026-07-21.csv
```

8. Back in the console, refresh the **Details** tab: **2,000,000** rows. Loads append by default; --replace would have truncated the table first.
9. Click **View all jobs** and look at the two most recent jobs: both are **Load** jobs, identical in kind. Console and CLI, same machinery.

Expected result. Table class.usage holds 2,000,000 rows, the job history shows two Load jobs, and nothing was billed — batch loads are free, and reading a bucket owned by another project is fine (you only need read access to the objects).

Gotcha box

- **gs:// in the console file field.** The console wants bucket/path with no gs:// prefix; pasting the full URI is the most common way this step "can't find the file."
- **Cloud Shell pointing at another project.** bq acts on whichever project Cloud Shell is configured for. If class "doesn't exist," check the project before re-creating anything.

Activity 2 (DIN §25) — Schemas: autodetect is a starting point (the pitfall)

What you'll do. Generate two dirty vendor files locally, load both with autodetect, and watch the same bad value cause a **loud failure** in one and a **silent failure** in the other. Then pin a schema, declare the vendor's null marker, and load cleanly.

Why it matters. Your migration will ingest vendor and partner files with exactly this class of dirt. The fix pattern — a pinned schema JSON in source control plus `--null_marker` — slots straight into your CI/CD discipline: the schema becomes a versioned artifact, not a console setting. Autodetect is the "Suggest Types" button: fine for a first look, never for a recurring pipeline.

Steps.

1. Open the local copy of `autodetect_demo.sh`, copy its entire contents, and paste them into your Cloud Shell terminal. It creates three files:
 - `daily_usage_mon.csv` — Monday's big drop (100,000 rows); the vendor writes N/A in the minutes column for meetings that never started; Monday's N/A sits at row 90,000
 - `daily_usage_tue.csv` — Tuesday's small drop (1,000 rows); its N/A is at row 700
 - `daily_usage_schema.json` — the pinned schema used by the fixed loads
2. Look at the first rows of Monday's file, and find the N/A rows in both files:

```
head -3 daily_usage_mon.csv
grep -n "N/A" daily_usage_mon.csv daily_usage_tue.csv
```

3. **The loud failure.** Load Monday's file, letting autodetect choose the types:

```
bq load --autodetect --source_format=CSV class.daily_usage daily_usage_mon.csv
```

The load **fails**. Read the error: "Unable to parse." Autodetect sampled the start of the file, saw only numbers in minutes, chose INTEGER — and row 90,000 said N/A.

4. **The silent failure.** Load Tuesday's file the same way:

```
bq load --autodetect --source_format=CSV class.daily_usage daily_usage_tue.csv
```

This load **succeeds**. Check what autodetect decided:

```
bq show --schema class.daily_usage
```

minutes is **STRING**. Tuesday's file is small enough that the N/A fell *inside* autodetect's sample, so it picked the only type that fits everything. Now try to use the data in the BigQuery editor:

```
SELECT SUM(minutes) AS total_minutes FROM class.daily_usage
```

The query fails: no version of SUM accepts a STRING. Same bad value, two different outcomes, decided only by where the value sat relative to the sample. Monday failed loudly — annoying, but you found out immediately. Tuesday failed silently — the load worked, the schema is wrong, and the damage surfaces later in every query that does math on minutes. **The silent one is the one that costs you days.**

5. **Pin the schema, name the marker.** Look at the schema file, then reload both days correctly — --replace on the first load (clear out Tuesday's silent-failure table), plain append on the second:

```
cat daily_usage_schema.json
```

```
bq load --replace --source_format=CSV \  
  --schema=daily_usage_schema.json \  
  --skip_leading_rows=1 \  
  --null_marker='N/A' \  
  class.daily_usage daily_usage_mon.csv
```

```
bq load --source_format=CSV \  
  --schema=daily_usage_schema.json \  
  --skip_leading_rows=1 \  
  --null_marker='N/A' \  
  class.daily_usage daily_usage_tue.csv
```

6. Verify in the BigQuery editor:

```
SELECT  
  COUNT(*) AS rows_loaded,  
  COUNTIF(minutes IS NULL) AS null_minutes,  
  SUM(minutes) AS total_minutes  
FROM class.daily_usage
```

Expected result. 101,000 rows loaded, exactly 2 NULL minutes (the meetings that never started), and SUM works.

Gotcha box

- **The Monday failure is the lesson, not a mistake.** If your load "breaks," you did it right — read the error message before moving on.
- **Don't drop --skip_leading_rows=1 once the schema is pinned.** Autodetect was quietly skipping the header for you; with an explicit schema you must say so yourself, or the header row fails against your INTEGER column.
- **Re-running appends.** Run the Tuesday load twice and you'll have 102,000 rows. Loads append by default — --replace exists for a reason.
- **Cheap path to a schema file.** Let autodetect run once on a known-good file, dump it with bq show --schema, review and edit, then load with --schema from then on.

Activity 3 (DIN §26) — Load performance: file formats matter (CSV vs Parquet race)

What you'll do. Sixty million rows of usage history sit in the demo bucket twice — once as a single gzipped CSV (~317 MiB), once as 100 Parquet files (~829 MiB total). You'll load both and compare the times.

Why it matters. This feeds directly into your Bronze-layer decision: what format do landing files arrive in, and what do you convert to? Wherever you control the format, land Parquet (or better). For vendor CSVs you *don't* control, the mitigation is many smaller uncompressed files over one large gzip. And notice the schema angle: Parquet carries real types in the file — the whole N/A-in-a-numeric-column failure class disappears when the format can't represent it.

Steps.

1. Look at both stagings, then place your bet — which loads faster?

```
gcloud storage ls -l gs://jwd-gcp-demos/ingest_demo/big/
```

```
gcloud storage du -s --readable-sizes gs://jwd-gcp-demos/ingest_demo/big_parquet
```

2. Load the gzipped CSV, with an explicit schema, and keep an eye on the elapsed-seconds counter bq prints while it waits:

```
bq load --source_format=CSV \
  --skip_leading_rows=1 \
  --schema=meeting_id:STRING,user_id:STRING,minutes:INTEGER,event_date:DATE \
  class.usage_history_csv \
  gs://jwd-gcp-demos/ingest_demo/big/usage_big.csv.gz
```

3. Around **two minutes** later it finishes. While you waited: a gzip file **cannot be split**, so one worker had to decompress and parse it from start to finish, alone. The 100 Parquet files can be handed to 100 workers at once, and Parquet is a binary columnar format, so there is no text parsing at all. That's how "more than twice the bytes" loads roughly 20x faster.
4. Load the Parquet copy of the same data. No --schema this time — Parquet files carry their own:

```
bq load --source_format=PARQUET \
  class.usage_history_pq \
  'gs://jwd-gcp-demos/ingest_demo/big_parquet/usage_big-*.parquet'
```

5. Confirm both tables:

```
bq show class.usage_history_csv
```

```
bq show class.usage_history_pq
```

Expected result. CSV load: about two minutes. Parquet load: seconds, despite being over twice the bytes. Both tables show **60,000,000** rows, and the Parquet-loaded table got the right types (INTEGER, DATE) from the files themselves.

Gotcha box

- **It is not hung.** Two minutes of waiting on the CSV load *is* the demonstration. Don't Ctrl-C — watch the elapsed counter and think about what one lonely worker is doing.
- **Quote the wildcard URI.** The single quotes around the Parquet path stop your shell from trying to expand the * itself — BigQuery expands it.
- **Don't pass --schema with Parquet.** The files know their own types; an inline schema is unnecessary noise.

Activity 4 (DIN §27) — LOAD DATA: loading with SQL

What you'll do. Re-do the activity-1 loads as pure SQL statements from `load_data_example.sql`, with the schema pinned in the column list.

Why it matters. A load expressed as SQL can live inside a script, a stored procedure, or a scheduled query — it hooks straight into the automation instincts you already have. One scheduled LOAD DATA with a date-built URI is the nightly-extract replacement in a single statement; the Day-1 closing demo schedules a query in one click, and you'll recognize the shape.

Steps.

1. Open `load_data_example.sql` and review the `-- load monday` query statement: the column list pins the schema the way `--schema` did on the CLI, and the `FROM FILES` options mirror the `bq load` flags. Run it in the BigQuery query editor. It finishes in a few seconds and creates the `usage_sql` table.

```
LOAD DATA INTO class.usage_sql (
  meeting_id STRING,
  user_id STRING,
  minutes INT64,
  event_date DATE
)
FROM FILES (
  format = 'CSV',
  skip_leading_rows = 1,
  uris = ['gs://jwd-gcp-demos/ingest_demo/daily/usage_2026-07-20.csv']
)
```

2. Review and run the `-- load tuesday` query statement: no column list this time, because the table now exists. `LOAD DATA INTO` appends; `LOAD DATA OVERWRITE` would replace.

```
LOAD DATA INTO class.usage_sql
FROM FILES (
  format = 'CSV',
  skip_leading_rows = 1,
  uris = ['gs://jwd-gcp-demos/ingest_demo/daily/usage_2026-07-21.csv']
)
```

3. Run the `-- verify` query statement:

```
SELECT
  event_date,
```

```
COUNT(*) AS meetings,
SUM(minutes) AS total_minutes
FROM class.usage_sql
GROUP BY event_date
ORDER BY event_date
```

Expected result. Each of the two dates has **1,000,000** meetings. Open **Job information** for either load: **Bytes billed: 0 B**. **LOAD DATA** is a SQL statement, but it runs the same free load machinery as the console form and bq load — which makes it the loading tool for places where everything must be SQL: scheduled queries, stored procedures, and scripts that create, load, and transform in one place.

Gotcha box

- **Run the statements in order, one at a time.** Monday's statement creates the table; Tuesday's depends on it existing. Run, wait, run.
- **usage_sql vs usage.** Activity 1's table is still around; these statements deliberately write a separate table so you can compare the two paths.
- **Free load, billed query.** Don't let "0 B" confuse you later: loads are free, queries bill by bytes scanned. The verify query is the only thing in this activity that could ever cost anything.

If you finish early

- Sketch the scheduled query that would load *today's* vendor file automatically: **LOAD DATA** with a URI built by **FORMAT_DATE**. Which column would you partition **usage_sql** by? Compare your sketch with the scheduled-query demo at the Day-1 close.
- Think through the vendor-change questions: what else do vendors put in numeric columns — empty strings, null, -, missing? What happens on the day the vendor adds a new column to the file?

Self-check

- ☐ **class.usage** shows **2,000,000** rows, and you can find both Load jobs in the job history.
- ☐ You saw Monday's autodetect load fail, and you can explain why (the N/A sat outside the sample, so autodetect chose **INTEGER** and the parse failed at row 90,000).
- ☐ You saw Tuesday's *silent* failure: minutes typed as **STRING**, and **SUM** failing on it.
- ☐ After the pinned-schema loads: **101,000** rows, exactly **2** **NULL** minutes, **SUM** works.
- ☐ **class.usage_history_csv** and **class.usage_history_pq** both show **60,000,000** rows — and you wrote down the two elapsed times.
- ☐ **class.usage_sql** shows **1,000,000** meetings per date, and Job information shows **Bytes billed: 0 B**.

What this replaces in your current stack

- **SSIS flat-file import packages** → bq load in a script, or **LOAD DATA** in SQL.
- **The Import and Export wizard** → console Create table from Google Cloud Storage.

- **SQL Agent import jobs** → a scheduled query running LOAD DATA (previewed at the Day-1 close).
- **Schema-drift surprises in production** → a pinned schema JSON reviewed in source control, plus --null_marker for vendor sentinels.

The rules you just learned

- **Pin your schemas.** Autodetect is a first-look tool; anything recurring gets an explicit schema, versioned like code.
- **Prefer Parquet for bulk.** Columnar, splittable, self-describing — minutes became seconds on the same 60 million rows.
- **Load, don't stream-insert, files.** Batch loads are free and built for files; row-by-row inserts are for live event streams, not vendor drops.
- **Everything is repeatable SQL.** Console clicks and CLI flags are the same job; the SQL form goes into source control, procedures, and schedules.

Part of the Iowa DOM DoIT Data Engineering student pack — see ../00_schedule.md.